

Week 1

Classificatie met Naive Bayes, KNN

Herhalen

Wat weten jullie hier nog over ?

- Classificatie vs Clustering
- Supervised (begeleid) vs Unsupervised (onbegeleid)
- Underfitting & Overfitting
- Train / Test / Validation data
- Cross-validatie

flowchart LR

Data["Dataset"] --> Split["Split 80/20"]

Split --> Train["Train set"]

Split --> Test["Test set"]

Train --> CV["Cross-validatie
(bv. 5-fold)"]

CV --> Model["Getraind model"]

Model --> Eval["Evalueren op test set"]

Scikit-learn

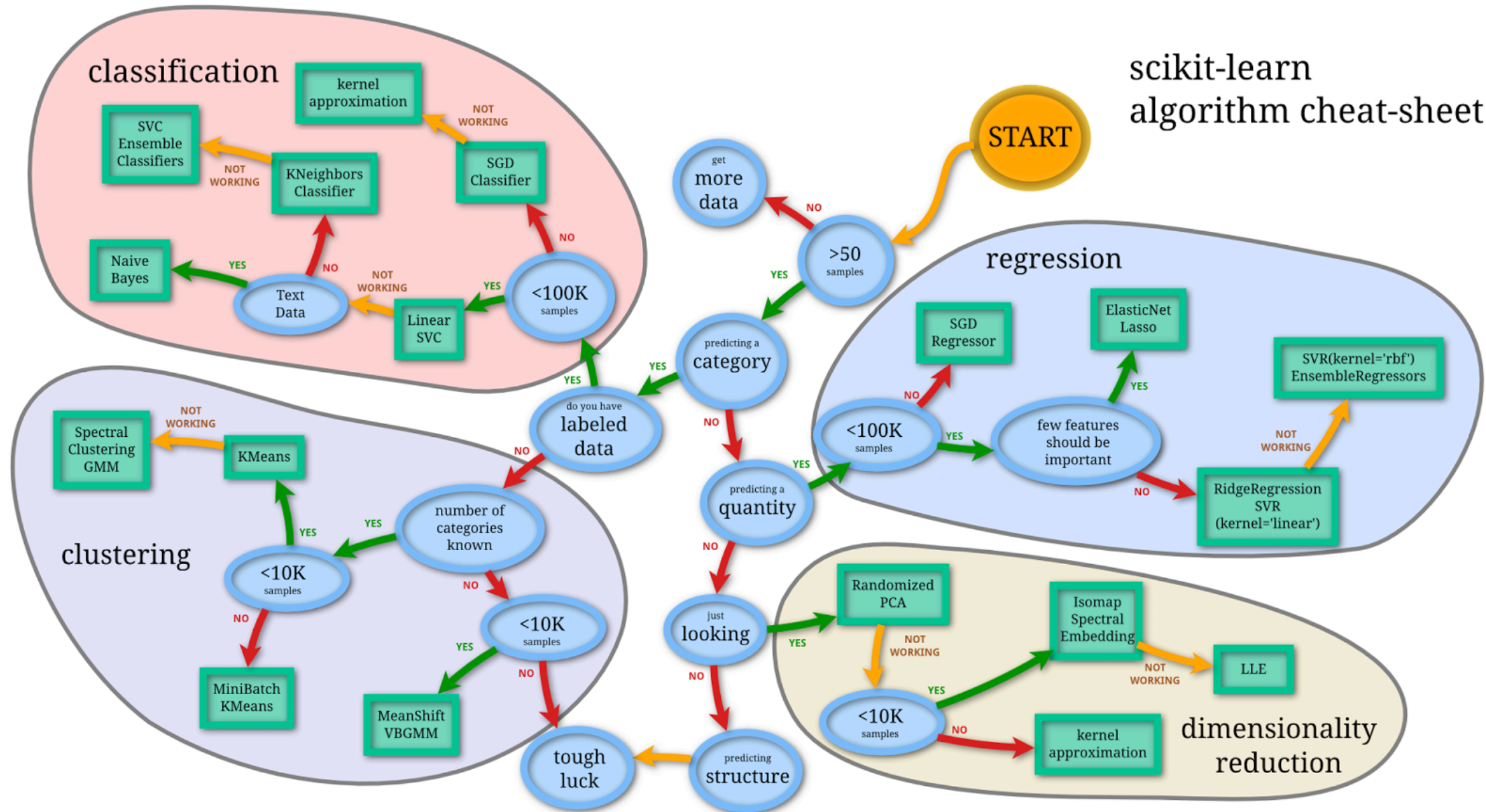
De standaard ML-bibliotheek voor Python

Wat is scikit-learn?

- **Bekendste ML-bibliotheek** voor Python — gebouwd op NumPy & SciPy
- **Wat kan het?**
 - Classificatie (o.a. Naive Bayes, KNN, SVM, ...)
 - Regressie
 - Clustering
 - Dimensionality reduction
 - Preprocessing & feature engineering
 - Model selectie & evaluatie (cross-validatie, metrics)

- **Open source** (BSD-licentie) — gratis, ook voor commercieel gebruik
 - Ontwikkeld door INRIA (Frans onderzoeksinstituut) en de community
 - Volledige broncode op GitHub
 - Actieve community, uitgebreide documentatie

scikit-learn algorithm cheat-sheet



Consistent API: één patroon voor elk algoritme

Elk schattingsalgoritme ("estimator") in scikit-learn volgt dezelfde structuur:

```
from sklearn.xxxx import SomeModel

model = SomeModel(hyperparameter=waarde) # 1. Maak model
model.fit(X, y) # 2. Leer uit data
model.predict(X_new) # 3. Voorspel
model.score(X_test, y_test) # 4. Evalueer
```

```
flowchart LR
    A["Model maken"] --> B["fit(X, y)"]
    B --> C["predict(X_new)"]
    C --> D["score(X_test, y_test)"]
```

Waarom? Eén patroon leren = **alle** algoritmes in deze cursus kunnen gebruiken!

... en nog meer herbruikbare methodes

Methode	Gebruik	Voorbeeld
<code>.fit(X, y)</code>	Train het model	<code>knn.fit(X_train, y_train)</code>
<code>.predict(X)</code>	Voorspel labels	<code>knn.predict(X_test)</code>
<code>.score(X, y)</code>	Bereken nauwkeurigheid	<code>knn.score(X_test, y_test)</code>
<code>.transform(X)</code>	Transformeer data (preprocessing)	<code>scaler.transform(X_test)</code>
<code>.fit_transform(X)</code>	Fit + transform in één stap	<code>scaler.fit_transform(X_train)</code>

Opgelet: `.fit()` en `.predict()` zijn **altijd** beschikbaar voor supervised modellen;
`.transform()` enkel voor preprocessing / unsupervised.

Naive Bayes

Een eerste classifieer

Naive Bayes — historische noot

- **Thomas Bayes (1701–1761)** — Engelse predikant en wiskundige
 - Zijn stelling werd **postuum** gepubliceerd in 1763 door zijn vriend Richard Price
- **Pierre-Simon Laplace (1749–1827)** — Franse wiskundige
 - Herontdekte en generaliseerde Bayes' werk onafhankelijk → **Bayes–Laplace-stelling**

- **Eerste ML-toepassing:** M.E. Maron (1961) gebruikte Naive Bayes voor automatische indexering van documenten
 - doi.org/10.1145/321075.321084
- **Spamfilter-revolutie (jaren 1990–2000):** Naive Bayes werd de standaard voor e-mail filtering
 - Sahami et al., 1998: aaai.org/Papers/Workshops/1998/WS-98-05/WS98-05-004.pdf
 - Graham, 2002: paulgraham.com/spam.html
- **Waarom "naief"?** — De aanname van onafhankelijkheid is al sinds de jaren 1960 gekend als *onrealistisch*, toch blijft de classifier verrassend goed presteren 🎯

Conditionele kans (voorwaardelijke kans)

- $P(A \mid B)$: kans op A als B al gebeurd is
- Definitie:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

- Voorbeeld — Dobbelsteen:
 - $P(\text{twee keer zes}) = 1/36 = P(A \cap B)$
 - $P(\text{é é n keer zes}) = 1/6 = P(B)$
 - $P(\text{twee keer zes} \mid \text{al é é n keer zes}) = \frac{1/36}{1/6} = \frac{1}{6}$

Regel van Bayes — formule

$$P(A \mid B) = \frac{P(B \mid A) P(A)}{P(B)}$$

Alternatieve vorm (handig in de praktijk):

$$P(A \mid B) = \frac{P(B \mid A) P(A)}{P(B \mid A)P(A) + P(B \mid \neg A)P(\neg A)}$$

Bayes Rule flow

```
flowchart LR
    A["P(B|A)"] --> C["P(B|A) · P(A)"]
    B["P(A)"] --> C
    C --> D["P(A|B)"]
    E["P(B)"] --> D
```


Regel van Bayes — in ML-context

- **B** = data / feiten (de e-mail)
- **A** = het classificatielabel dat we willen toekennen (spam / ham)

We willen dus berekenen:

$$P(\text{label} \mid \text{data})$$

"Wat is de kans op dit label, **gegeven** de data die we zien?"

Spamfilter — probleemstelling

We bouwen een **spamfilter** dat e-mails labelt als 'spam' of 'geen spam'.

Gegeven:

- $P(\text{moneytransfer} \mid \text{spam}) = 0.4$ (40% van spam bevat 'moneytransfer')
- $P(\text{moneytransfer} \mid \neg \text{spam}) = 0.0001$ (0.01% van ham bevat het)
- $P(\text{spam}) = 0.45$ (45% van alle e-mail is spam)

Vraag: Een e-mail bevat 'moneytransfer' — is het spam?

flowchart TD

```
Email["Email ontvangen"] --> Check{"Bevat 'moneytransfer'?"}  
Check -- Ja --> Bereken["Bereken  $P(\text{spam}|\text{mt})$   
en  $P(\neg\text{spam}|\text{mt})$ "]  
Check -- Nee --> Next["Check volgende woorden"]  
Bereken --> Kies{"Welke kans is groter?"}  
Kies -- Spam --> LabelS[" SPAM"]  
Kies -- Ham --> LabelH[" HAM"]
```

Moneytransfer → Spam?

Berekening via Bayes:

$$\begin{aligned} P(\text{spam} \mid \text{mt}) &= \frac{P(\text{mt} \mid \text{spam}) P(\text{spam})}{P(\text{mt} \mid \text{spam}) P(\text{spam}) + P(\text{mt} \mid \neg \text{spam}) P(\neg \text{spam})} \\ &= \frac{0.4 \times 0.45}{0.4 \times 0.45 + 0.0001 \times 0.55} \\ &\approx 0.9997 \end{aligned}$$

Besluit: $P(\text{spam} \mid \text{mt}) \approx 99.97\% \rightarrow \text{label} = \text{spam}$ 

Meerdere woorden screenen — gegevens

Woord	$P(\text{woord} \mid \text{spam})$	$P(\text{woord} \mid \neg \text{spam})$
'moneytransfer'	0.4	0.0001
'uganda'	0.1	0.05
'AP Hogeschool'	0.001	0.3

Vraag: Een e-mail bevat **niet** 'moneytransfer', maar **wel** 'uganda' en 'AP Hogeschool'.
Wat is de kans op spam?

Meerdere datapunten — berekening

Stap 1 — Bayes toepassen:

$$P(\text{spam} \mid \neg\text{mt}, \text{ug}, \text{ap}) = \frac{P(\neg\text{mt}, \text{ug}, \text{ap} \mid \text{spam}) \cdot P(\text{spam})}{P(\neg\text{mt}, \text{ug}, \text{ap})}$$

Stap 2 — Naïeve aanname: woorden zijn onafhankelijk

$$P(\neg\text{mt}, \text{ug}, \text{ap} \mid \text{spam}) = P(\neg\text{mt} \mid \text{spam}) \times P(\text{ug} \mid \text{spam}) \times P(\text{ap} \mid \text{spam})$$

Stap 3 — Samengevoegd voor spam:

$$P(\text{spam} \mid \neg\text{mt}, \text{ug}, \text{ap}) \propto P(\neg\text{mt} \mid \text{spam}) \times P(\text{ug} \mid \text{spam}) \times P(\text{ap} \mid \text{spam}) \times P(\text{spam})$$

Stap 4 — Idem voor geen spam:

$$P(\neg\text{spam} \mid \neg\text{mt}, \text{ug}, \text{ap}) \propto P(\neg\text{mt} \mid \neg\text{spam}) \times P(\text{ug} \mid \neg\text{spam}) \times P(\text{ap} \mid \neg\text{spam}) \times P(\neg\text{spam})$$

Uiteindelijk: kies het label met **grootste teller** — de noemer $P(\text{data})$ is voor beide gelijk, dus hoeft je die niet uit te rekenen!

Naive Bayes Classifier

Waarom "naief"?

- De kansen van woorden **samen** en **apart** zijn niet onafhankelijk — in werkelijkheid beïnvloeden ze elkaar.
- Toch werkt deze classifier **in de praktijk verrassend goed**, dus we negeren dit bezwaar.

Laplace-smoothing

Als $P(x_i \mid y = k) = 0$ voor een label, wordt het hele product 0.

Oplossing: tel een kleine waarde δ op:

$$P(x_i \mid y = k) = \frac{\delta + \#(x_i, y = k)}{\sum_j (\delta + \#(x_j, y = k))}$$

- δ wordt bepaald via cross validation

Implementatie in sklearn

```
from sklearn.naive_bayes import BernoulliNB  
  
clf = BernoulliNB()  
clf.fit(X, Y)  
clf.predict(X[2:3])
```

Volgende stap: het trainen van de data flexibeler maken → **Labo!**

K-Nearest Neighbours (KNN)

KNN — historische noot

- **Fix & Hodges (1951)** — Eerste beschrijving van het "nearest neighbor"-beslissingsregel
 - US Air Force technisch rapport: een **niet-parametrische** classifieer — geen aannames over de onderliggende verdeling
 - Zie: en.wikipedia.org/wiki/K-nearest_neighbors_algorithm
- **Cover & Hart (1967)** — Baanbrekend artikel "Nearest neighbor pattern classification"

- Bewezen: met voldoende data is de fout van 1-NN **hoogstens $2\times$ de Bayes-optimale fout**
- doi.org/10.1109/TIT.1967.1053964
- "**Lazy learning**" — al in de jaren 1950 gekend: geen echte trainingsfase, alle data wordt opgeslagen
- **1990–2010** — KNN breed toegepast in patroonherkenning, beeldclassificatie en aanbevelingssystemen (collaborative filtering)

 KNN is één van de **oudste** en **simpelste** ML-algoritmes — maar nog steeds relevant!

Wat bepaalt de prijs van een huis?

Locatie, locatie en nog eens locatie!

flowchart LR

Huis["🏠 Nieuw huis
(prijs = ?)"] --> B1["📍 Buur 1: €350k"]

Huis --> B2["📍 Buur 2: €320k"]

Huis --> B3["📍 Buur 3: €370k"]

Huis --> B4["📍 Buur 4: €315k"]

B1 & B2 & B3 & B4 --> Gem["Gemiddelde ≈ €339k"]

Gem --> Prijs["→ Geschatte prijs: €339k"]

KNN — Basisidee

Gelijkaardige dingen zijn dicht bij elkaar.

Algoritme:

1. Kies K = aantal buren om rekening mee te houden
 2. Voor een nieuw punt:
 - Bereken de K dichtste datapunten
 - Het **meest voorkomende label** van die K punten = voorspelling
-  Training kost nauwelijks tijd — maar **classificatie is traag** (veel afstanden berekenen)

flowchart TD

Start["Nieuw datapunt x"] --> K["Kies K (aantal buren)"]

K --> Afstand["Bereken afstand tot
alle train-data punten"]

Afstand --> Sorteert["Sorteer op afstand (↑)"]

Sorteer --> Selecteer["Selecteer K dichtste buren"]

Selecteer --> Stem["Stem: meest voorkomende
label van K buren"]

Stem --> Predict["Voorspel label voor x"]

KNN — Eigenschappen (1)

- **Supervised learning** — zowel binary als multiclass
- Werkt **enkel** als er een notie van **afstand** is tussen datapunten
- **Numerieke data**: normaliseren is **verplicht!**
 - Anders krijgen features met grote waarden onbedoeld meer gewicht
- **Categorische data**: zelf een metriek definiëren
 - Bv. vogelvluchtafstand tussen steden, of **one-hot encoding**

KNN — Eigenschappen (2)

- K moet op voorhand gekozen worden
 - Te kleine K → gevoelig voor ruis (overfitting)
 - Te grote K → te veel middeling (underfitting)
 - **Oplossing:** kies K via **cross-validatie**
- **Training:** goedkoop (enkel data opslaan) ✓
- **Classificatie:** duur (alle afstanden berekenen) ✗

KNN is een **lazy learner** — er is geen echte trainingsfase.

KNN — Visueel (2 kolommen)

flowchart LR

A[" Punt 1
(dichtstbij)"] -->|"k=5"| N{" ? Nieuw
punt"}

B[" Punt 2"] --> N

C[" Punt 3"] --> N

D[" Punt 4"] --> N

E[" Punt 5
(verst)"] --> N

N --> Result["Stemming: 4x, 1x"]

Result --> Predict["→ Voorspelling: "]

- $k = 5$
- Het algoritme vindt 4 rood en 1 groen label
- ➡ Voorspelling voor het nieuwe punt = rood

Implementatie met Python & sklearn

Optie 1 — Zelf implementeren (zie labo)

Optie 2 — Scikit-learn:

```
from sklearn.neighbors import KNeighborsClassifier  
  
knn = KNeighborsClassifier(n_neighbors=5)  
knn.fit(X_train, y_train)  
predictions = knn.predict(X_test)
```

Belangrijk!

-  Normaliseer features **vóór** KNN
-  Kies K via cross-validatie, niet "op het gevoel"
-  Gebruik `.fit` / `.predict` — dit patroon komt overal terug in sklearn